



Departamento de
Informática

Universidade da Beira Interior
Sistemas Operativos — 2025/2026

Simulador de Gestão de Processos

Escalonamento e Gestão de Memória

Elemento 1: Tiago Marques — a48950

Elemento 2: Tiago Valente — a48234

Repositório: <https://github.com/tiagolv/S0Projeto>

24 de maio de 2026

Conteúdo

1	Introdução	2
2	Descrição Geral do Simulador	3
2.1	Modelo de Processo	3
2.2	Contexto de um Processo (PCB)	3
2.3	Modelo de Memória	4
2.4	Estruturas de Dados Globais	5
3	Arquitetura Modular	6
4	Funcionamento Detalhado	7
4.1	Inicialização	7
4.2	Gestor de Processos	7
4.3	Execução de um Quantum	7
4.4	Preempção	8
4.5	Ciclo de Vida de um Processo	8
5	Algoritmos de Escalonamento	9
5.1	Escalonamento de Curto Prazo	9
5.1.1	FCFS — <i>First Come, First Served</i>	9
5.1.2	PRIo — Prioridade	9
5.1.3	SJF — <i>Shortest Job First</i>	9
5.1.4	RM — <i>Rate Monotonic</i>	9
5.1.5	EDF — <i>Earliest Deadline First</i>	9
5.2	Escalonamento de Longo Prazo	9
6	Gestão de Memória	10
6.1	Protótipos	10
6.2	Deduplicação	10
6.3	Compactação	10
7	Protótipos das Funções	11
8	Exemplos de Execução e Verificação	13
8.1	Ficheiros de Exemplo	13
8.2	Verificação — FCFS com quantum 4	13
8.3	Verificação — SJF com quantum 4	15
8.4	Exemplo de Modo Interativo	15
9	Compilação e Execução	16
10	Conclusão	17

1 Introdução

O presente relatório descreve o desenvolvimento de um simulador de gestão de processos realizado no âmbito da unidade curricular de Sistemas Operativos do 2.º semestre de 2025/2026, na Universidade da Beira Interior.

O objetivo do projeto é simular, em linguagem C, o comportamento de um sistema operativo no que diz respeito à gestão de processos: criação, execução, bloqueio, terminação e comutação de contexto, bem como os algoritmos de escalonamento do processador (CPU scheduling).

O simulador foi desenvolvido de forma modular, com um *Makefile* que permite compilar toda a aplicação com um único comando.

2 Descrição Geral do Simulador

Esta secção descreve os elementos fundamentais do simulador: o modelo de processo adotado, a estrutura de memória partilhada, o *Process Control Block* e as variáveis globais que constituem o estado do sistema.

2.1 Modelo de Processo

Cada processo executa um programa constituído por uma sequência de instruções simples que operam sobre uma única variável inteira. As instruções suportadas são as seguintes:

Tabela 1: Conjunto de instruções do simulador

Instrução	Descrição
M <i>n</i>	Atribui o valor <i>n</i> à variável do processo
A <i>n</i>	Adiciona <i>n</i> à variável
S <i>n</i>	Subtrai <i>n</i> à variável
B	Bloqueia o processo (move-o para a fila de bloqueados)
T	Termina o processo
C <i>n</i>	<i>Fork</i> : cria um processo filho (cópia do pai); o filho continua na instrução seguinte e o pai salta <i>n</i> instruções
L <i>filename</i>	<i>Exec</i> : substitui o programa atual pelo ficheiro indicado

Cada instrução é executada em exatamente **1 unidade de tempo** de simulação.

2.2 Contexto de um Processo (PCB)

O estado de cada processo é descrito por um *Process Control Block* (PCB), que contém os campos listados na Tabela 2.

Tabela 2: Campos do PCB

Campo	Descrição
pid	Identificador do processo
ppid	Identificador do processo pai
nome	Nome do ficheiro de programa
start	Índice de início do programa na memória
size	Número de instruções do programa
pc	Contador do programa (<i>Program Counter</i>)
var	Valor atual da variável inteira do processo
priority	Prioridade (menor valor = maior prioridade)
state	Estado: READY, RUNNING, BLOCKED, TERMINATED
start_time	Tempo de criação do processo
cpu_time	Tempo total de CPU consumido
end_time	Tempo de terminação
period	Período (para escalonamento em tempo real)
deadline	Prazo absoluto (para EDF)

2.3 Modelo de Memória

Todos os programas em execução são carregados num único array global de instruções com 1000 posições:

```

1 typedef struct { char ins; int n; char nome[16]; } instruction;
2 instruction memory[1000];

```

Listing 1: Estrutura da memória

A gestão desta memória inclui:

- **Alocação:** procura do primeiro bloco contíguo livre suficientemente grande para o programa a carregar.
- **Libertação:** quando um processo termina e nenhum outro processo partilha o mesmo bloco, as posições são limpas.
- **Deduplicação:** se um programa já estiver carregado em memória por outro processo ativo, reutiliza-se o mesmo bloco em vez de carregar um duplicado.
- **Compactação** (*mem_compact*): quando a alocação falha por falta de espaço contíguo (apesar de existir espaço total suficiente), os blocos vivos são movidos para o início da memória e todos os PCBs são atualizados em conformidade.

2.4 Estruturas de Dados Globais

Tabela 3: Principais variáveis globais do simulador

Variável	Descrição
tempo	Relógio global de simulação (unidades de tempo)
running	Índice do PCB do processo em execução (-1 se vazio)
time_quantum	Quantum de tempo máximo por execução
policy	Política de escalonamento ativa
ready_q	Fila de processos prontos
blocked_q	Fila de processos bloqueados
term_q	Fila de processos terminados
pcb_table	Tabela de PCBs (máx. 50 processos)
memory	Array de instruções (memória do sistema)

3 Arquitetura Modular

O simulador segue uma organização modular em que cada ficheiro é responsável por uma preocupação bem definida. A Tabela 4 resume os módulos existentes e as suas responsabilidades.

O código está organizado nos módulos descritos na Tabela 4.

Tabela 4: Módulos do simulador

Ficheiro	Responsabilidade
<code>sim.c / sim.h</code>	Inicialização, ciclo principal e despacho de comandos
<code>process.c / process.h</code>	Ciclo de vida dos processos (criação, fork, exec, bloqueio, terminação)
<code>memory.c / memory.h</code>	Gestão do array de memória (alocação, libertação, compactação)
<code>loader.c / loader.h</code>	Leitura de ficheiros de programa, <code>plan.txt</code> e <code>control.txt</code>
<code>scheduler.c / scheduler.h</code>	Escalonamento de curto e longo prazo
<code>exec.c / exec.h</code>	Execução de instruções e lógica do quantum
<code>queue.c / queue.h</code>	Fila circular genérica com remoção arbitrária
<code>report.c / report.h</code>	Relatórios periódicos e estatísticas globais
<code>main.c</code>	Ponto de entrada, parsing de argumentos

4 Funcionamento Detalhado

Esta secção descreve o fluxo de execução do simulador desde a inicialização até à terminação, passando pelo ciclo de execução de instruções, pelo mecanismo de preempção e pelo ciclo de vida completo de um processo.

4.1 Inicialização

A função `sim_init()` inicializa todas as estruturas de dados (PCB, filas, memória), define o quantum e a política por omissão (FCFS com quantum 4) e inicializa o gerador de números aleatórios com `srand(time(NULL))`.

De seguida, `sim_run()` lê o ficheiro `plan.txt`, ordena as entradas por tempo de chegada (`qsort`), e processa o ficheiro de controlo.

4.2 Gestor de Processos

O gestor lê comandos do ficheiro `control.txt` (ou do *stdin* em modo interativo). Os comandos disponíveis são:

Tabela 5: Comandos do gestor

Comando	Efeito
E	Executa um quantum de tempo (chama <code>exec_quantum</code>)
I	Interrompe e bloqueia o processo em execução
D	Escalonador de longo prazo: desbloqueia aleatoriamente um processo
R	Imprime o estado atual do sistema
T	Termina o simulador e imprime as estatísticas globais

4.3 Execução de um Quantum

A função `exec_quantum()` executa até `time_quantum` instruções do processo corrente. Em cada iteração:

1. Se não houver processo em execução, chama o escalonador de curto prazo.
2. Executa uma instrução e incrementa `cpu_time` e `tempo`.
3. Verifica chegada de novos processos (`sim_check_arrivals`).
4. Verifica se o processo terminou ou foi bloqueado.
5. Verifica se deve ocorrer preempção (ver Secção 4.4).

No final do quantum, se o processo ainda estiver em execução, é movido de volta para a fila de prontos.

4.4 Preempção

O simulador suporta escalonamento preemptivo. Após cada instrução, a função `should_preempt()` verifica se existe na fila de prontos um processo com maior prioridade segundo a política ativa:

- **FCFS**: não preemptivo dentro do quantum.
- **PRIO**: preemptivo se existir processo com prioridade numericamente inferior (maior precedência).
- **SJF**: preemptivo se existir processo com menos instruções restantes.
- **RM**: preemptivo se existir processo com período mais curto.
- **EDF**: preemptivo se existir processo com prazo mais próximo.

4.5 Ciclo de Vida de um Processo

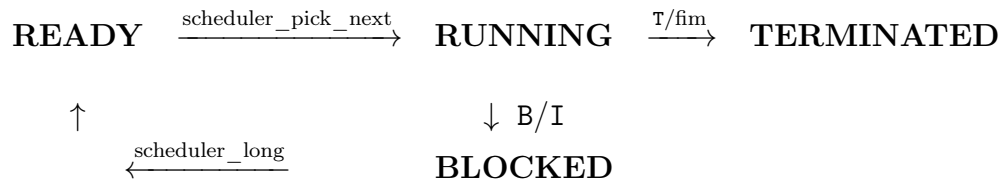


Figura 1: Diagrama simplificado de estados de um processo

5 Algoritmos de Escalonamento

Foram implementados cinco algoritmos de escalonamento de curto prazo, todos suportando preempção, bem como um escalonador de longo prazo para gestão de processos bloqueados. A política ativa é selecionável como argumento na linha de comando.

5.1 Escalonamento de Curto Prazo

Foram implementados cinco algoritmos, selecionáveis por argumento da linha de comando:

5.1.1 FCFS — *First Come, First Served*

Seleciona o processo que está há mais tempo na fila de prontos. Implementado como uma fila FIFO simples. É não preemptivo dentro do quantum.

5.1.2 PRIO — Prioridade

Seleciona o processo com menor valor de prioridade (menor número = maior prioridade). É preemptivo: se um processo com maior prioridade chegar durante a execução, o processo corrente é recolocado na fila de prontos imediatamente.

5.1.3 SJF — *Shortest Job First*

Seleciona o processo com menos instruções restantes ($size - (pc - start)$). Implementado de forma preemptiva (SRTN — *Shortest Remaining Time Next*).

5.1.4 RM — *Rate Monotonic*

Atribui prioridade estática inversamente proporcional ao período do processo. Seleciona sempre o processo com menor período. Preemptivo.

5.1.5 EDF — *Earliest Deadline First*

Seleciona sempre o processo com o prazo absoluto mais próximo. Preemptivo e ótimo para sistemas de tempo real.

5.2 Escalonamento de Longo Prazo

A função `scheduler_long()` é responsável por desbloquear processos. Utiliza `rand()` para escolher aleatoriamente um processo da fila de bloqueados e movê-lo para a fila de prontos, simulando a ocorrência de um evento externo que desbloqueia o processo.

6 Gestão de Memória

Todos os programas em execução partilham um único array de instruções de 1000 posições. Esta secção descreve as operações de alocação, libertação, deduplicação e compactação que garantem a utilização eficiente desse espaço.

6.1 Protótipos

```
1 void mem_init(void);
2 void mem_compact(void);
3 int mem_load_program(const char *filename,
4                       int *start_out, int *size_out);
5 void mem_clear_range(int start, int size);
6 int mem_find_free(int size);
```

Listing 2: Protótipos de memory.h

6.2 Deduplicação

Antes de carregar um programa, o sistema verifica se esse programa já está presente em memória (através da função `find_loaded_program`). Em caso afirmativo, o processo partilha o mesmo bloco de memória, evitando duplicados.

6.3 Compactação

Se `mem_find_free` não encontrar espaço contíguo suficiente, é invocada `mem_compact()`, que:

1. Identifica todos os segmentos de programa ativos.
2. Ordena-os por endereço de início.
3. Usa `memmove` para os compactar para o início do array.
4. Atualiza os campos `start` e `pc` de todos os PCBs afetados.
5. Zera a região libertada no final do array.

7 Protótipos das Funções

Listam-se abaixo os protótipos públicos de cada módulo, organizados por ficheiro de cabeçalho. Não é incluído código fonte, conforme indicado no enunciado.

```
1 void sim_init(void);
2 void sim_run(const char *plan_file, const char *control_file);
3 void sim_tick(void);
4 void sim_check_arrivals(void);
5 void sim_interrupt_running(void);
```

Listing 3: sim.h

```
1 int process_create(const char *prog, int ppid, int priority);
2 int process_fork(int parent_index, int jump);
3 int process_exec(int idx, const char *prog);
4 void process_block(int idx);
5 void process_ready(int idx);
6 void process_terminate(int idx);
```

Listing 4: process.h

```
1 int scheduler_pick_next(void);
2 void scheduler_long(void);
3 const char *scheduler_name(sched_policy p);
```

Listing 5: scheduler.h

```
1 void mem_init(void);
2 void mem_compact(void);
3 int mem_load_program(const char *filename,
4                      int *start_out, int *size_out);
5 void mem_clear_range(int start, int size);
6 int mem_find_free(int size);
```

Listing 6: memory.h

```
1 int loader_read_program(const char *filename,
2                          instruction *buf, int max,
3                          int *out_count);
4 int loader_read_plan(const char *filename);
5 int loader_read_control(const char *filename);
```

Listing 7: loader.h

```
1 void exec_quantum(void);
```

Listing 8: exec.h

```
1 void queue_init(queue_t *q);  
2 int queue_is_empty(queue_t *q);  
3 int queue_is_full(queue_t *q);  
4 int queue_push(queue_t *q, int value);  
5 int queue_pop(queue_t *q);  
6 int queue_peek(queue_t *q);  
7 int queue_remove(queue_t *q, int value);
```

Listing 9: queue.h

```
1 void report_state(void);  
2 void report_global(void);
```

Listing 10: report.h

8 Exemplos de Execução e Verificação

Esta secção apresenta dois exemplos de execução com verificação manual dos resultados, utilizando os programas de exemplo incluídos no repositório, seguidos de um exemplo do modo interativo.

8.1 Ficheiros de Exemplo

Os programas de exemplo utilizados são os seguintes:

Tabela 6: Programas de exemplo

p1.prg	filho1.prg	filho2.prg
M 100	M 200	M 300
A 19	A 19	S 12
A 20	T	A 5
S 12		T
A 1		
A 4		
C 2		
L filho1		
C 2		
L filho2		
T		

8.2 Verificação — FCFS com quantum 4

Ficheiro plan.txt:

1	p1.prg	0	2
2	p1.prg	10	1
3	filho1.prg	20	3

Listing 11: plan.txt

Ficheiro control.txt: Ficheiro control.txt:

1	E
2	E
3	R
4	E
5	D
6	E
7	T
8	

Figura 2: control.txt

Resultado esperado e verificação do var:

$M 100 \rightarrow 100$, $+19 \rightarrow 119$, $+20 \rightarrow 139$, $-12 \rightarrow 127$, $+1 \rightarrow 128$, $+4 \rightarrow 132$

Após 8 ticks (2 quantum de 4), o processo pai executou as 6 primeiras instruções de `p1.prg` e duas iterações do C/L, criando os filhos 2 e 3 com `var = 132`.

Saída real do simulador (extrato do comando R a tempo=8, seguido das estatísticas globais):

```
tiagolv17@LAPTOP-DN869CPC:/mnt/c/Users/Tiago/Documents/GitHub/SOProjeto/soubi-master/simulador$ ./sim
PROCESSOS BLOQUEADOS:
PROCESSOS PRONTOS:
  [p1.prg] pid=2 ppid=1 prio=2 var=132 start_t=6 cpu=0
  [p1.prg] pid=3 ppid=1 prio=2 var=132 start_t=7 cpu=0
  [p1.prg] pid=1 ppid=0 prio=2 var=132 start_t=0 cpu=8
PROCESSOS TERMINADOS:

=== ESTATISTICAS GLOBAIS (tempo=16, escalonador=FCFS) ===
PROCESSOS TERMINADOS:
  [filho1.prg] pid=2 ppid=1 prio=2 var=219 start_t=6 cpu=4 end_t=11 turnaround=5
Media turnaround=5.0 Media cpu=4.0 Processos=1
```

Figura 3: Saída completa — FCFS quantum=4

5.0 Media cpu=4.0 Processos=1

Verificação de filho1.prg:

$M 200 \rightarrow 200$, $+19 \rightarrow 219$, T

O filho 2 (pid=2) começa no tempo 6, executa 3 instruções com 1 quantum de espera, termina em `end_t=11`, `turnaround = 11 - 6 = 5`.

8.3 Verificação — SJF com quantum 4

Com o escalonador SJF e 5 quantum, `filho2.prg` (4 instruções, o menor *burst* disponível) é selecionado antes dos processos com mais instruções restantes. O `p1.prg` original (`pid=1`) termina primeiro pois tem menos instruções restantes após os forks.

```
tiagolv17@LAPTOP-DN869CPC:/mnt/c/Users/Tiago/Documents/GitHub/SOProjeto/soubi-master/simulador$ ./simulador plan.txt control_sjf.txt 4 SJF
TEMPO ACTUAL: 14
ESCALONADOR: SJF
PROCESSO EM EXECUCAO:
(nenhum)
PROCESSOS BLOQUEADOS:
PROCESSOS PRONTOS:
[p1.prg] pid=2 ppid=1 prio=2 var=132 start_t=6 cpu=0
[p1.prg] pid=4 ppid=0 prio=1 var=0 start_t=10 cpu=0
PROCESSOS TERMINADOS:
[p1.prg] pid=1 ppid=0 prio=2 var=132 start_t=0 cpu=9 end_t=8 turnaround=8
[filho2.prg] pid=3 ppid=1 prio=2 var=293 start_t=7 cpu=5 end_t=13 turnaround=6

=== ESTATISTICAS GLOBAIS (tempo=14, escalonador=SJF) ===
PROCESSOS TERMINADOS:
[p1.prg] pid=1 ppid=0 prio=2 var=132 start_t=0 cpu=9 end_t=8 turnaround=8
[filho2.prg] pid=3 ppid=1 prio=2 var=293 start_t=7 cpu=5 end_t=13 turnaround=6
Media turnaround=7.0 Media cpu=7.0 Processos=2
```

Figura 4: Saída completa — SJF quantum=4 (5 quantums)

Verificação: $M300 \rightarrow 300$, $-12 \rightarrow 288$, $+5 \rightarrow 293$. Saída obtida: `var=293`.

8.4 Exemplo de Modo Interativo

```
1 echo -e "E\nE\nR\nT" | ./simulador plan.txt - 4 PRIO
```

Listing 12: Modo stdin (comando '-')

Este modo é ativado automaticamente quando o segundo argumento é `-` ou quando o ficheiro de controlo não existe.

9 Compilação e Execução

O projeto compila com um único comando `make` e não tem dependências externas para além de um compilador C compatível com POSIX. Os passos abaixo são suficientes para compilar e executar o simulador em qualquer sistema Linux, macOS ou Windows WSL.

```
1 cd simulador
2 make
```

Listing 13: Compilação

```
1 ./simulador [plan.txt] [control.txt] [quantum] [FCFS|PRIO|SJF|RM
|EDF]
```

Listing 14: Execução

Por omissão: `plan.txt`, `control.txt`, `quantum=4`, política FCFS.

10 Conclusão

O simulador desenvolvido cobre todos os requisitos obrigatórios do enunciado: suporta os sete tipos de instrução, gere o ciclo de vida completo de processos com operações de *fork* e *exec*, implementa cinco algoritmos de escalonamento (FCFS, Prioridade, SJF, RM e EDF) de forma preemptiva, gere a memória partilhada com deduplicação e compactação automática, e produz relatórios de estado e estatísticas globais com turnaround médio.

As decisões de projeto mais relevantes foram:

- A separação clara entre escalonamento de curto e longo prazo.
- A implementação de preempção genérica em `exec_quantum`, evitando duplicação de lógica entre algoritmos.
- A compactação de memória como mecanismo transparente de último recurso.
- O modo interativo via *stdin*, que facilita o teste e a depuração.